

- 东软智慧云脑诊疗平台 — 系统设计文档
 - 目录
 - 1. 系统概述
 - 2. 系统架构设计
 - 2.1 系统上下文架构
 - 2.2 技术架构全景
 - 2.3 前端架构设计
 - 2.4 后端分层架构
 - 2.5 项目源码结构
 - 3. 功能模块分解
 - 3.1 模块总览
 - 3.2 模块职责矩阵
 - 3.3 模块依赖关系
 - 4. 数据库设计
 - 4.1 实体关系总览 ER 图
 - 4.2 核心表结构设计
 - 模块一相关
 - 模块二相关
 - 模块三相关
 - 模块五/六相关
 - 5. 核心模块详细设计
 - 5.1 模块一：智能分诊与挂号模块
 - 5.1.1 模块概述
 - 5.1.2 类图
 - 5.1.3 核心时序图
 - 5.2 模块二：AI 病历生成模块
 - 5.2.1 模块概述
 - 5.2.2 类图
 - 5.2.3 核心时序图
 - 5.3 模块三：处方审核与开具模块
 - 5.3.1 模块概述
 - 5.3.2 类图
 - 5.3.3 核心时序图
 - 5.4 模块四：管理端基础数据模块
 - 5.4.1 模块概述
 - 5.4.2 类图
 - 5.4.3 核心时序图
 - 5.5 模块五：用户认证与通用基础设施模块
 - 5.5.1 模块概述
 - 5.5.2 类图
 - 5.5.3 核心时序图
 - 5.6 模块六：P1 进阶能力模块

- 5.6.1 模块概述
- 5.6.2 类图
- 5.6.3 核心时序图
- 5.6.4 其他 P1 能力概要设计
- 6. API 接口总览
 - 模块五——认证与患者管理
 - 模块一——智能分诊与挂号
 - 模块二——AI 病历生成
 - 模块三——处方审核与开具
 - 模块四——管理端
 - 模块六——P1 进阶能力
 - 统一响应格式
- 7. AI 服务集成设计
 - 7.1 AI Provider 抽象层
 - 7.2 处方审核双引擎架构
- 8. 安全设计

东软智慧云脑诊疗平台 — 系统设计文档

版本：v2.0 日期：2026-06-14 关联文档：PRD v1.2 | 需求.md

v2.0 变更说明：

- 修复全部 22 张 Mermaid 图渲染问题：纯 ASCII 节点 ID、双引号包裹文本、移除实验性语法、移除孤立节点
- 新增第 6 模块覆盖 P1 进阶能力（AI 诊疗建议、就诊评价、数据看板、SSE、状态机、Prompt 模板）
- 补充 F10-F11 医生列表/详情公开 API 至模块一
- ER 图移除未定义的 AuditLog，所有实体完整定义

目录

- 1. 系统概述
- 2. 系统架构设计
 - 2.1 系统上下文架构
 - 2.2 技术架构全景
 - 2.3 前端架构设计
 - 2.4 后端分层架构
 - 2.5 项目源码结构
- 3. 功能模块分解

- 3.1 模块总览
- 3.2 模块职责矩阵
- 3.3 模块依赖关系
- 4. 数据库设计
 - 4.1 实体关系总览 ER 图
 - 4.2 核心表结构设计
- 5. 核心模块详细设计
 - 5.1 模块一：智能分诊与挂号模块
 - 5.2 模块二：AI 病历生成模块
 - 5.3 模块三：处方审核与开具模块
 - 5.4 模块四：管理端基础数据模块
 - 5.5 模块五：用户认证与通用基础设施模块
 - 5.6 模块六：P1 进阶能力模块
- 6. API 接口总览
- 7. AI 服务集成设计
- 8. 安全设计

1. 系统概述

东软智慧云脑诊疗平台是一套基于 Spring Boot 3 + Vue 3 前后端分离架构的**三端智能门诊诊疗原型系统**，面向高校软件工程实训课程。系统覆盖患者端（手机窗口式）、医生端（PC 工作台）、管理端（PC 后台），融合 AI 大模型与本地处方规则引擎，实现"诊前分诊 → 线上挂号 → 医生接诊 → 病历生成 → 处方审核 → 患者查看"的一期诊疗闭环。

设计目标：

- 前后端分离，RESTful API + JWT 认证
- 后端 Controller-Service-Repository 三层分层
- AI 能力封装为独立 Service，支持多供应商切换
- 处方审核采用"本地规则优先 + 大模型解释"架构
- 三端共享同一后端 API，通过角色鉴权区分权限
- P0 核心模块 5 个 + P1 进阶模块 1 个，共 6 个功能模块

2. 系统架构设计

2.1 系统上下文架构

Parse error on line 1:
flowchart TB sub

```
^
Expecting 'NEWLINE', 'SPACE', 'GRAPH', got 'ALPHA'
```

2.2 技术架构全景

```
Parse error on line 1:
flowchart TB      sub
^
Expecting 'NEWLINE', 'SPACE', 'GRAPH', got 'ALPHA'
```

2.3 前端架构设计

```
Parse error on line 1:
flowchart TB      sub
^
Expecting 'NEWLINE', 'SPACE', 'GRAPH', got 'ALPHA'
```

三端路由设计：

端	路由前缀	示例
患者端	/patient/	/patient/login、/patient/triage、/patient/registrations
医生端	/doctor/	/doctor/workbench、/doctor/consultation/:id、/doctor/prescription/:id
管理端	/admin/	/admin/departments、/admin/schedules、/admin/drugs

2.4 后端分层架构

```
Parse error on line 1:
flowchart TB      sub
^
Expecting 'NEWLINE', 'SPACE', 'GRAPH', got 'ALPHA'
```

统一响应结构 **Result** (泛型 **<T>**):

```
public class Result<T> {
    private int code;           // 200 success, 4xx client error, 5xx server error
    private String message;
    private T data;
    private long timestamp;
}
```

2.5 项目源码结构

```
cloud-brain-medical/
├── backend/
│   ├── pom.xml
│   └── src/main/java/com/cloudbrain/
│       ├── config/                # SecurityConfig CorsConfig Knife4jConfig AIConfig
│       ├── security/              # JwtTokenUtil JwtAuthenticationFilter
│       ├── common/                # Result GlobalExceptionHandler BaseEntity
│       ├── controller/
│       │   ├── patient/           # Patient public controllers
│       │   ├── doctor/            # Doctor controllers
│       │   └── admin/             # Admin controllers
│       ├── service/
│       │   ├── business/          # Business services
│       │   └── ai/                # AI capability services
│       ├── repository/            # JPA repositories
│       ├── entity/                # JPA entities
│       ├── dto/                  # Request/Response DTOs
│       └── enums/                 # Enumerations
├── frontend/
│   ├── package.json
│   ├── vite.config.ts
│   └── src/
│       ├── api/                  # Axios API wrappers
│       ├── router/               # Vue Router config
│       ├── stores/               # Pinia stores
│       ├── views/
│       │   ├── patient/          # Patient pages
│       │   ├── doctor/           # Doctor pages
│       │   └── admin/            # Admin pages
│       ├── components/           # Shared components
│       └── utils/                # Utilities
└── docs/                         # Project documentation
```

3. 功能模块分解

3.1 模块总览

本系统划分为 **6 个功能模块**。其中模块一至五对应 P0 核心交付，模块六覆盖 P1 进阶能力。

Parse error on line 1:
flowchart TB sub
^
Expecting 'NEWLINE', 'SPACE', 'GRAPH', got 'ALPHA'

3.2 模块职责矩阵

模块	负责人	主要职责	包含 PRD 功能编号	P0 接口
模块一：智能分诊与挂号	成员 A	AI 分诊、公开科室/医生查询、排班号源查询、挂号与取消	F10-F19, F22	10 个
模块二：AI 病历生成	成员 B	问诊录入、AI 病历生成、病历编辑保存、病历列表查询	F31-F35	6 个
模块三：处方审核与开具	成员 C	处方开具、本地规则+大模型处方审核、审核记录、处方列表	F23, F25-F30	7 个
模块四：管理端基础数据	成员 D	科室/医生管理、排班号源、药品库 CRUD、处方规则 CRUD、AI 配置	F20-F24	18 个
模块五：认证与通用基础设施	成员 E	JWT 认证、三端登录注册、统一响应结构、全局异常、CORS、Swagger、联调配置	F01-F09, F36-F39	8 个
模块六：P1 进阶能力	先完成者	AI 诊疗建议、就诊评价与反馈、数据可视化看板、SSE 流式、Pinia 状态机、Prompt 模板	E01-E06	8 个

3.3 模块依赖关系

Parse error on line 1:
flowchart TB N_M
^
Expecting 'NEWLINE', 'SPACE', 'GRAPH', got 'ALPHA'

4. 数据库设计

4.1 实体关系总览 ER 图

Parse error on line 1:
erDiagram Patien

^
Expecting 'NEWLINE', 'SPACE', 'GRAPH', got 'ALPHA'

4.2 核心表结构设计

模块一相关

表名	核心字段	索引
patient	id, username, password_hash, phone, name, gender, birth_date, allergy_history, medical_history	UK(username), INDEX(phone)
department	id, code, name, type, status	UK(code)
doctor	id, username, password_hash, name, department_id, title, specialty, status	FK(department_id)
schedule	id, doctor_id, department_id, work_date, period, total_slots, remaining_slots	FK(doctor_id, department_id), INDEX(work_date)
registration	id, patient_id, doctor_id, department_id, schedule_id, registration_time, status	FK(patient_id, doctor_id, schedule_id)
triage_record	id, patient_id, chief_complaint, recommended_dept, recommended_doctors(JSON), call_status	FK(patient_id)

模块二相关

表名	核心字段	索引
medical_record	id, patient_id, doctor_id, registration_id, chief_complaint, present_illness, past_history, physical_exam, preliminary_diagnosis, treatment_plan, conversation_text, ai_generated	FK(patient_id, doctor_id, registration_id)

模块三相关

表名	核心字段	索引
prescription	id, patient_id, doctor_id, registration_id, risk_level, status	FK(patient_id, doctor_id, registration_id)

表名	核心字段	索引
prescription_item	id, prescription_id, drug_id, drug_name, specification, dosage, frequency, duration, quantity	FK(prescription_id, drug_id)
prescription_review	id, prescription_id, risk_level, local_rule_hits(JSON), llm_suggestion, llm_summary, call_status	FK(prescription_id)
drug	id, code, name, specification, dosage_form, contraindications, interaction_summary, status	UK(code), INDEX(name)
prescription_rule	id, drug_id, rule_type, risk_level, trigger_condition, alert_message, suggestion, regulation_reference	FK(drug_id)

模块五/六相关

表名	核心字段	索引
admin	id, username, password_hash, name, role	UK(username)
ai_config	id, provider, model_name, api_url, api_key_encrypted, timeout_seconds, status	—
feedback	id, patient_id, registration_id, rating, triage_accurate, comment	FK(patient_id, registration_id)

5. 核心模块详细设计

5.1 模块一：智能分诊与挂号模块

5.1.1 模块概述

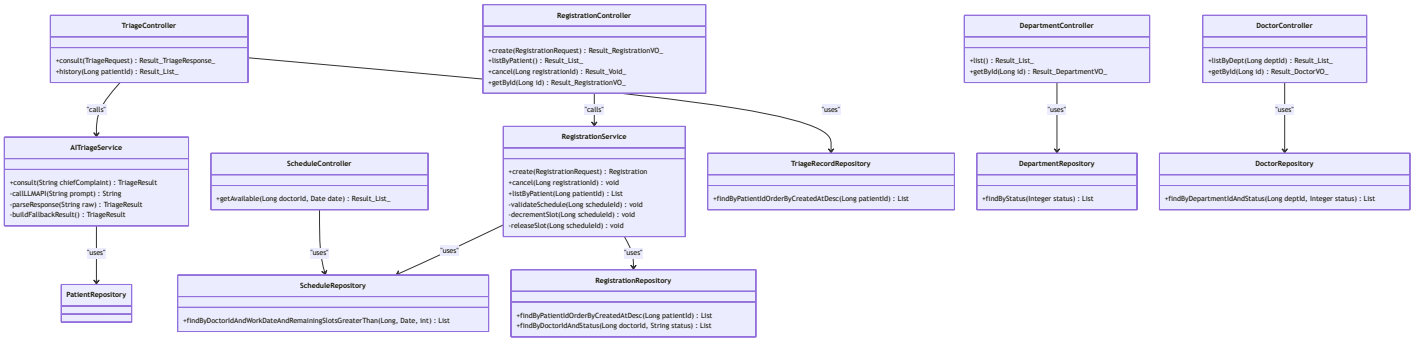
负责人：成员 A

职责：

- AI 分诊：接收患者主诉 → 调用大模型 → 推荐科室与医生 → 持久化分诊记录
- 公开查询：科室列表/详情、医生列表/详情（供患者端浏览，无需登录）
- 号源查询：按医生 + 日期查询可用排班号源
- 挂号管理：创建挂号（扣减号源）、查询我的挂号、取消挂号（释放号源）

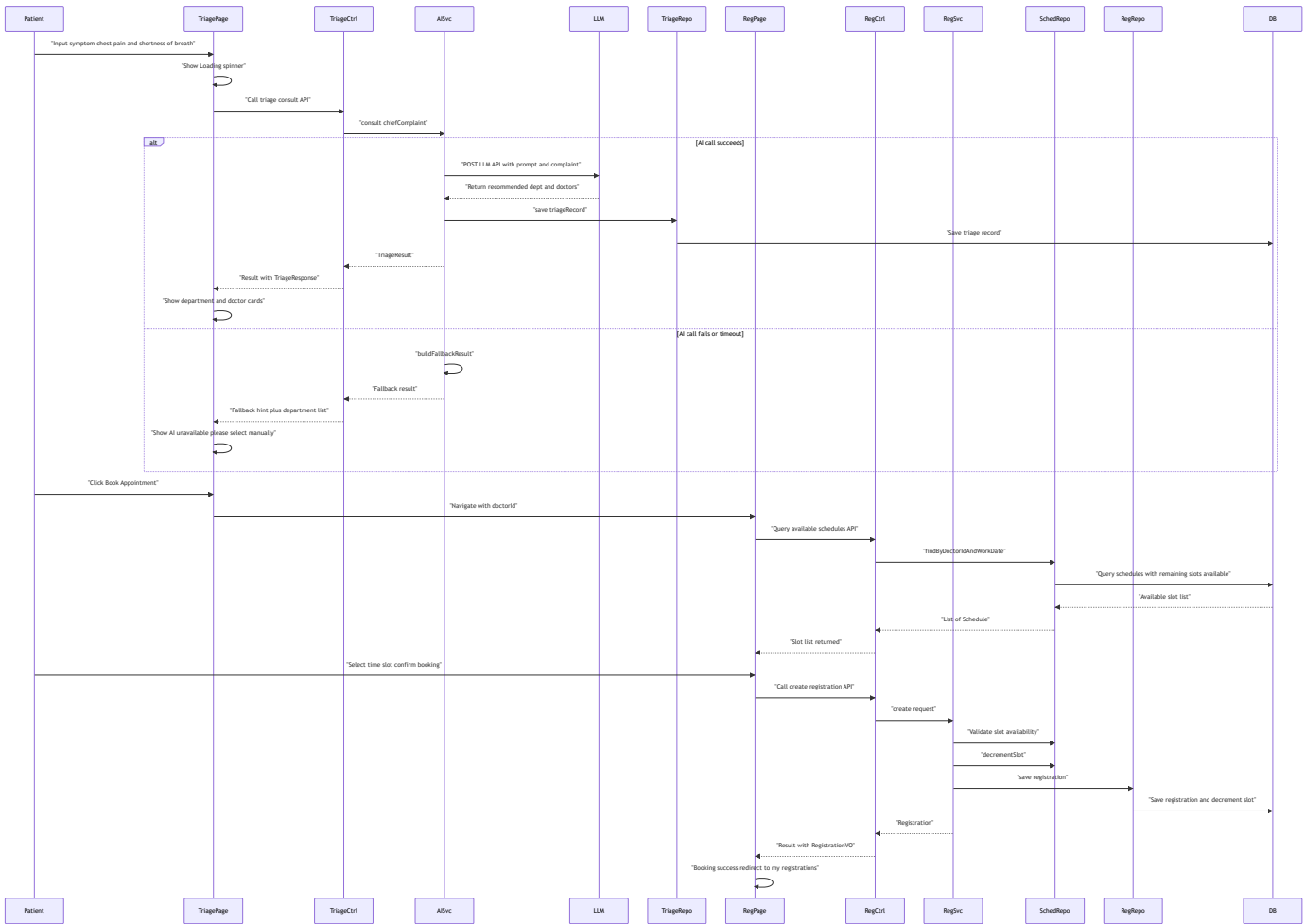
依赖：模块五（认证基础设施）、模块四（科室/医生/排班数据）

5.1.2 类图

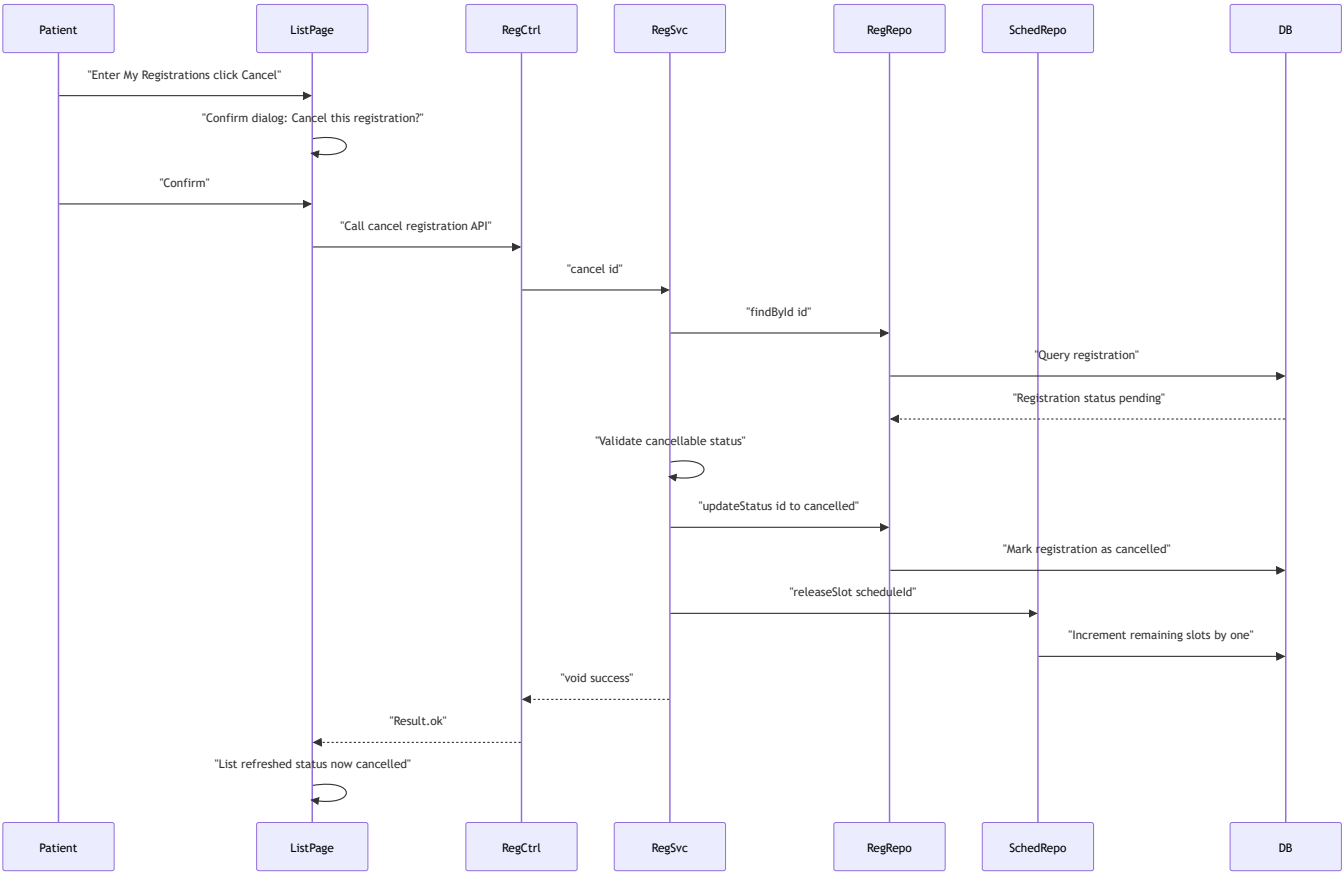


5.1.3 核心时序图

流程 A：智能分诊与挂号



流程 B：取消挂号（号源释放）



5.2 模块二：AI 病历生成模块

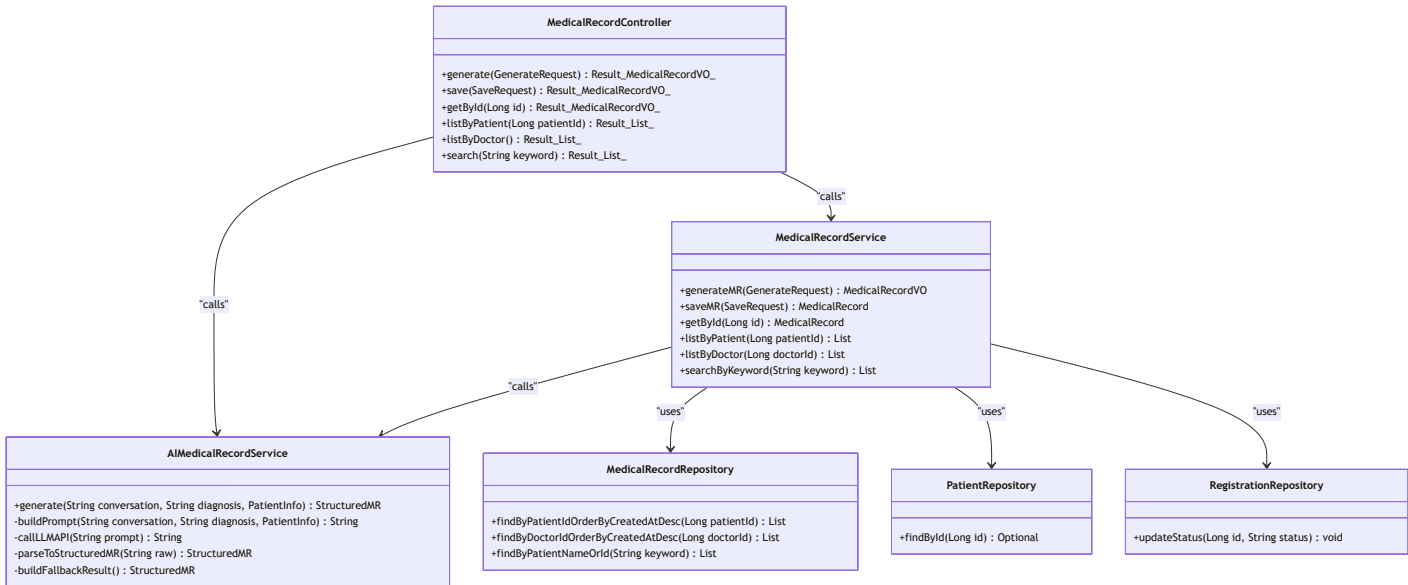
5.2.1 模块概述

负责人：成员 B

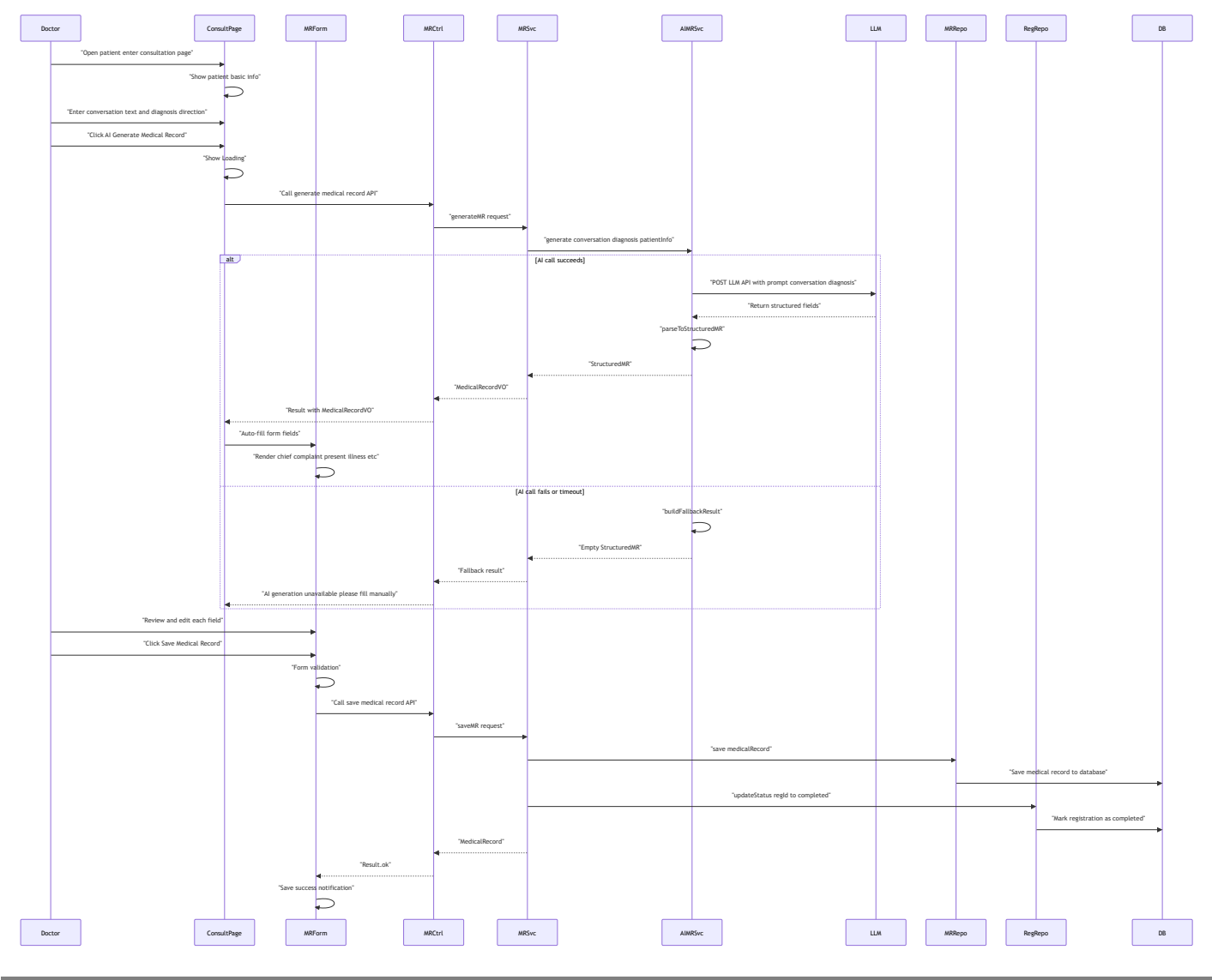
职责：问诊对话录入 → 调用 AI 病历生成引擎 → 结构化字段回填表单 → 医生编辑确认 → 保存病历 → 标记挂号完成。提供病历列表与按患者搜索功能。

依赖：模块五（认证）、模块一（挂号信息）

5.2.2 类图



5.2.3 核心时序图



5.3 模块三：处方审核与开具模块

5.3.1 模块概述

负责人：成员 C

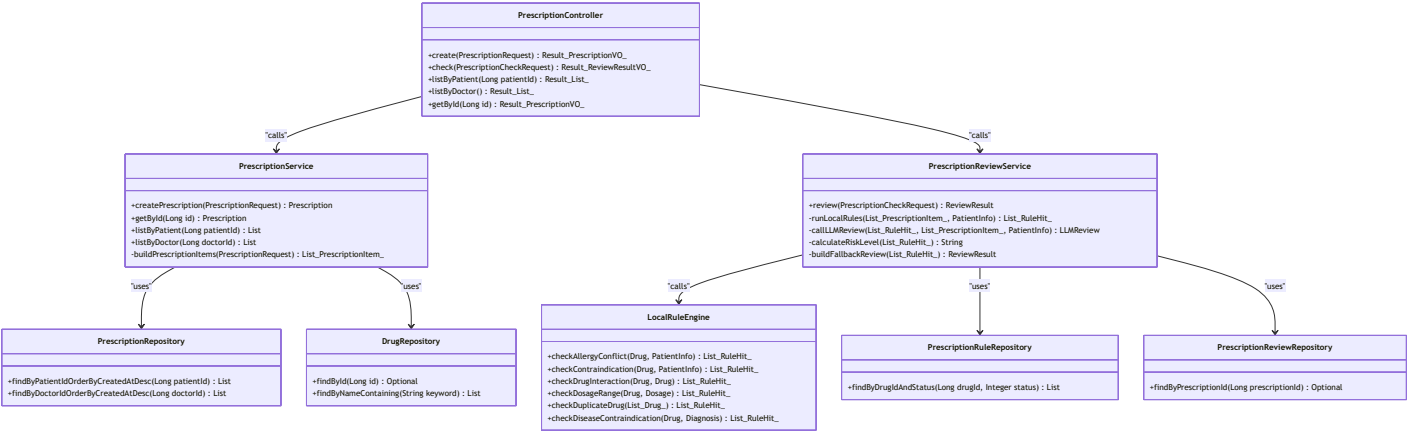
职责：从药品库选择药品 → 开具处方 → 本地规则引擎审核 → 大模型解释补充 → 审核结果展示 → 医生调整 → 提交保存。处方列表与审核记录查询。

核心设计——"本地规则优先 + 大模型解释"：

- 处方提交
- 本地规则引擎 查禁忌 相互作用 剂量范围 → 命中规则列表
 - 大模型 基于命中规则 处方上下文 生成解释建议 → 自然语言解释
 - 合并返回 规则命中项 风险等级 大模型解释

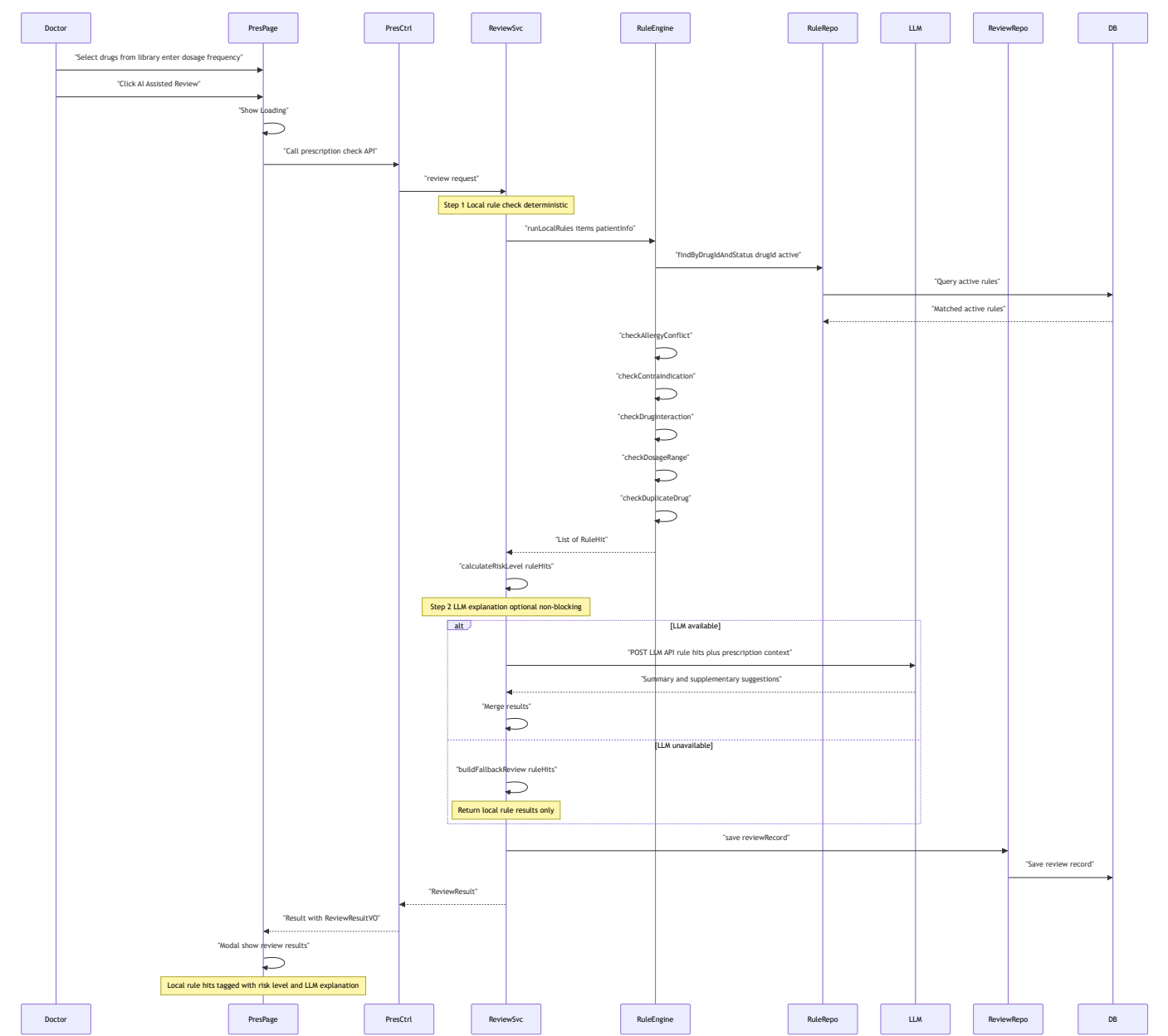
依赖：模块五（认证）、模块四（药品库/处方规则）、模块二（患者诊断信息）

5.3.2 类图

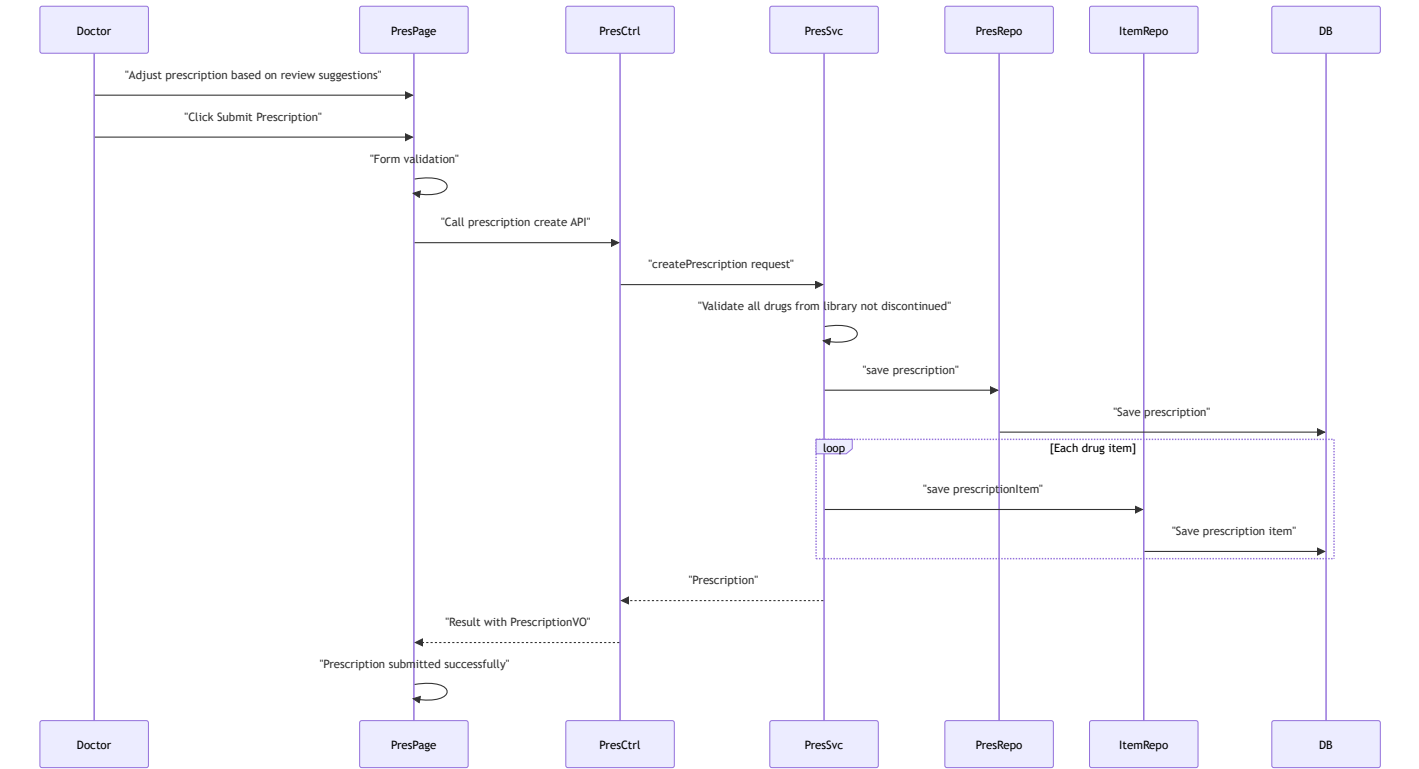


5.3.3 核心时序图

流程 A：处方审核（本地规则 + 大模型）



流程 B：处方提交保存



5.4 模块四：管理端基础数据模块

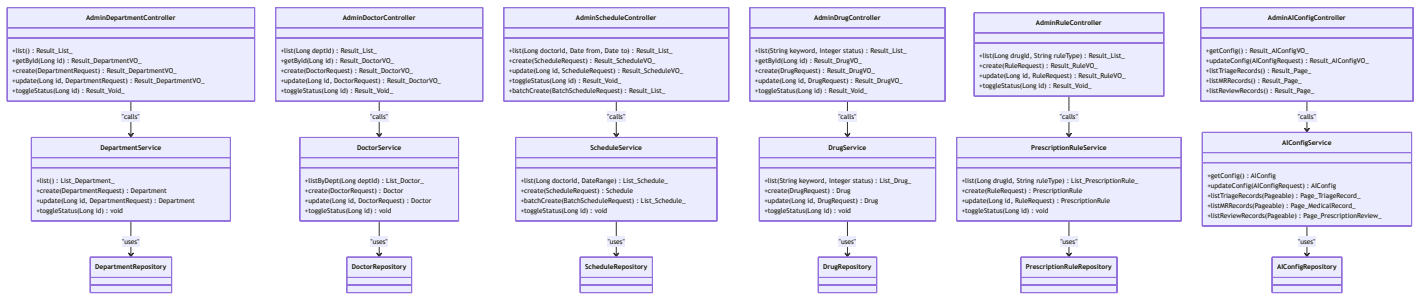
5.4.1 模块概述

负责人：成员 D

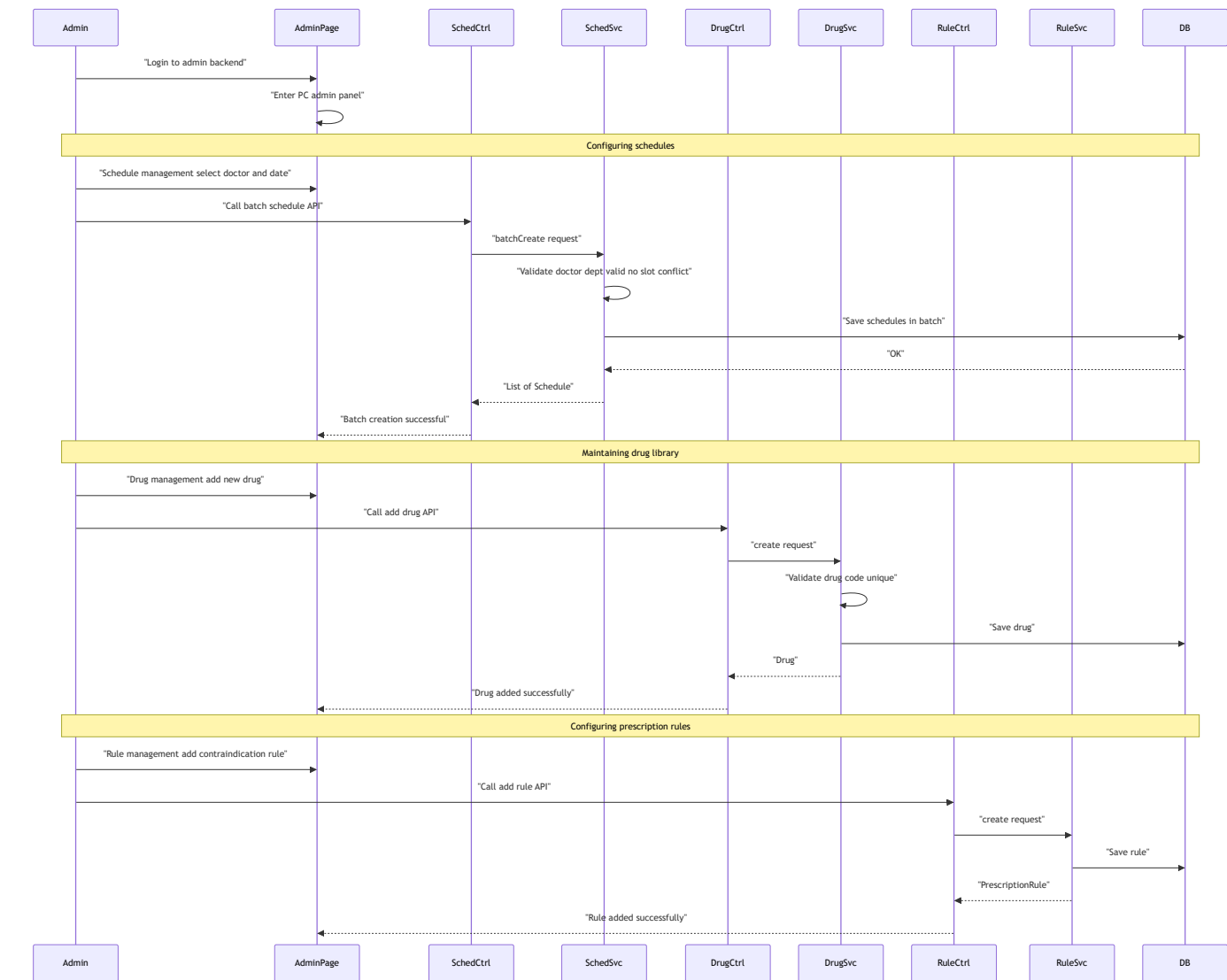
职责：管理端全部基础数据维护——科室 CRUD、医生 CRUD、排班号源 CRUD 及批量创建、药品库 CRUD、处方审核规则 CRUD、AI 服务配置、AI 调用记录查看。

被依赖：模块一（科室/医生/排班）、模块三（药品库/规则）

5.4.2 类图



5.4.3 核心时序图



5.5 模块五：用户认证与通用基础设施模块

5.5.1 模块概述

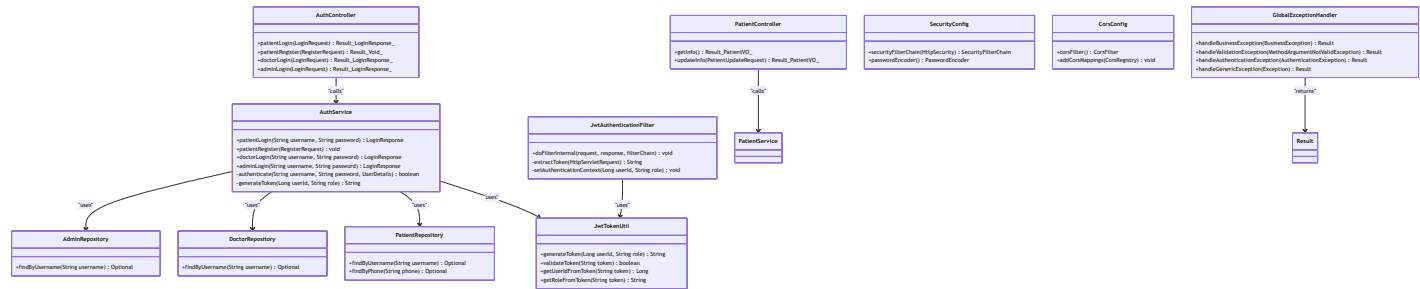
负责人：成员 E（最先完成，为其他模块提供基础）

职责：

- 三端认证：患者注册/登录、医生登录、管理员登录，JWT 签发与验证
- 患者信息查询与更新
- 通用基础设施：统一响应结构 Result、全局异常处理器、CORS 跨域配置、Swagger/Knife4j 文档
- 前端基础设施：Vite 项目创建、Axios 封装（请求/响应拦截器）、Vite 代理配置
- 联调支撑：JWT 自动携带、401 拦截跳转、跨域问题处理

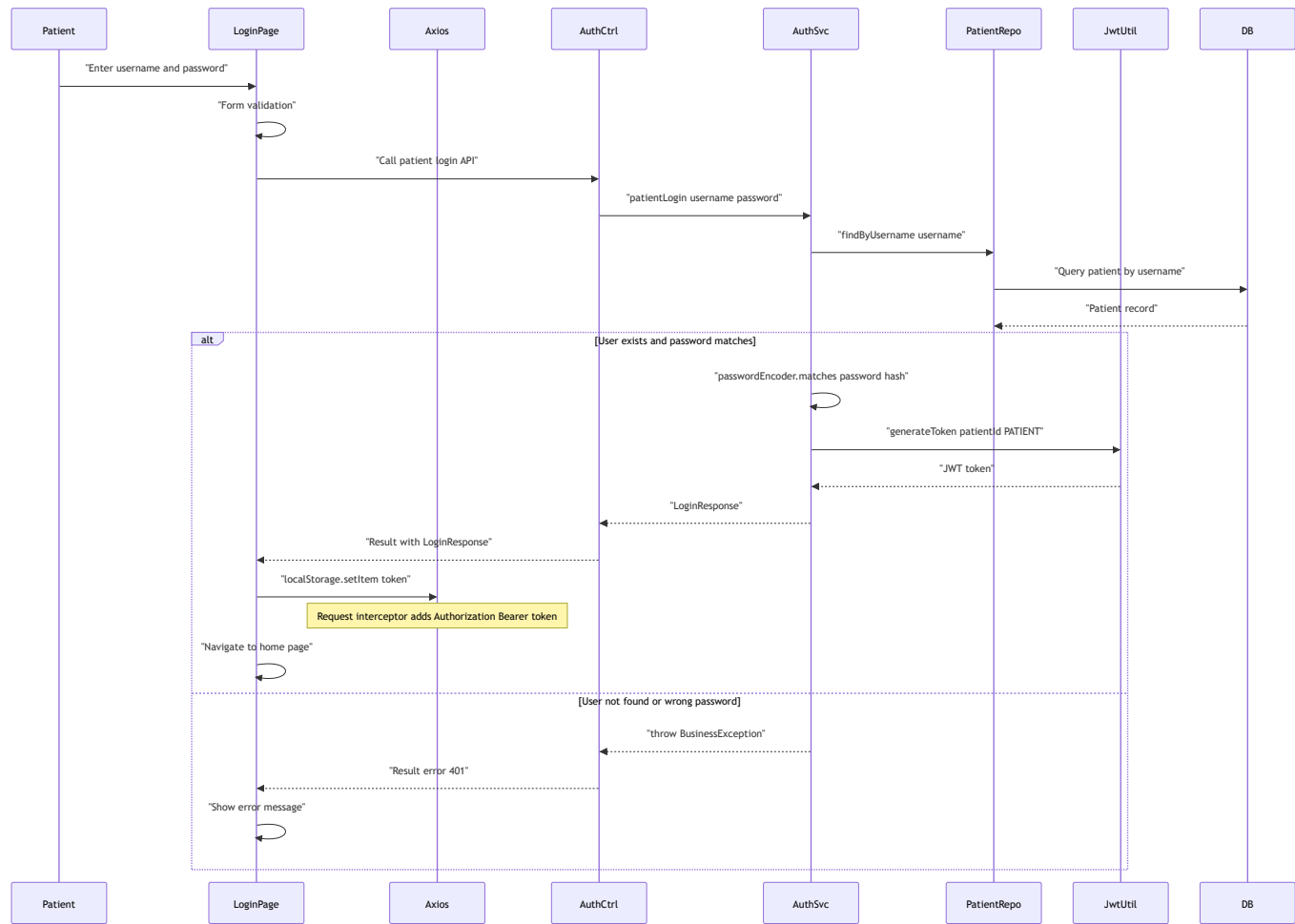
前置任务（项目初始化 F01-F06）： Spring Boot 3 + Vue 3 项目脚手架、MySQL 配置、Knife4j、CORS、统一响应结构、Axios 封装

5.5.2 类图

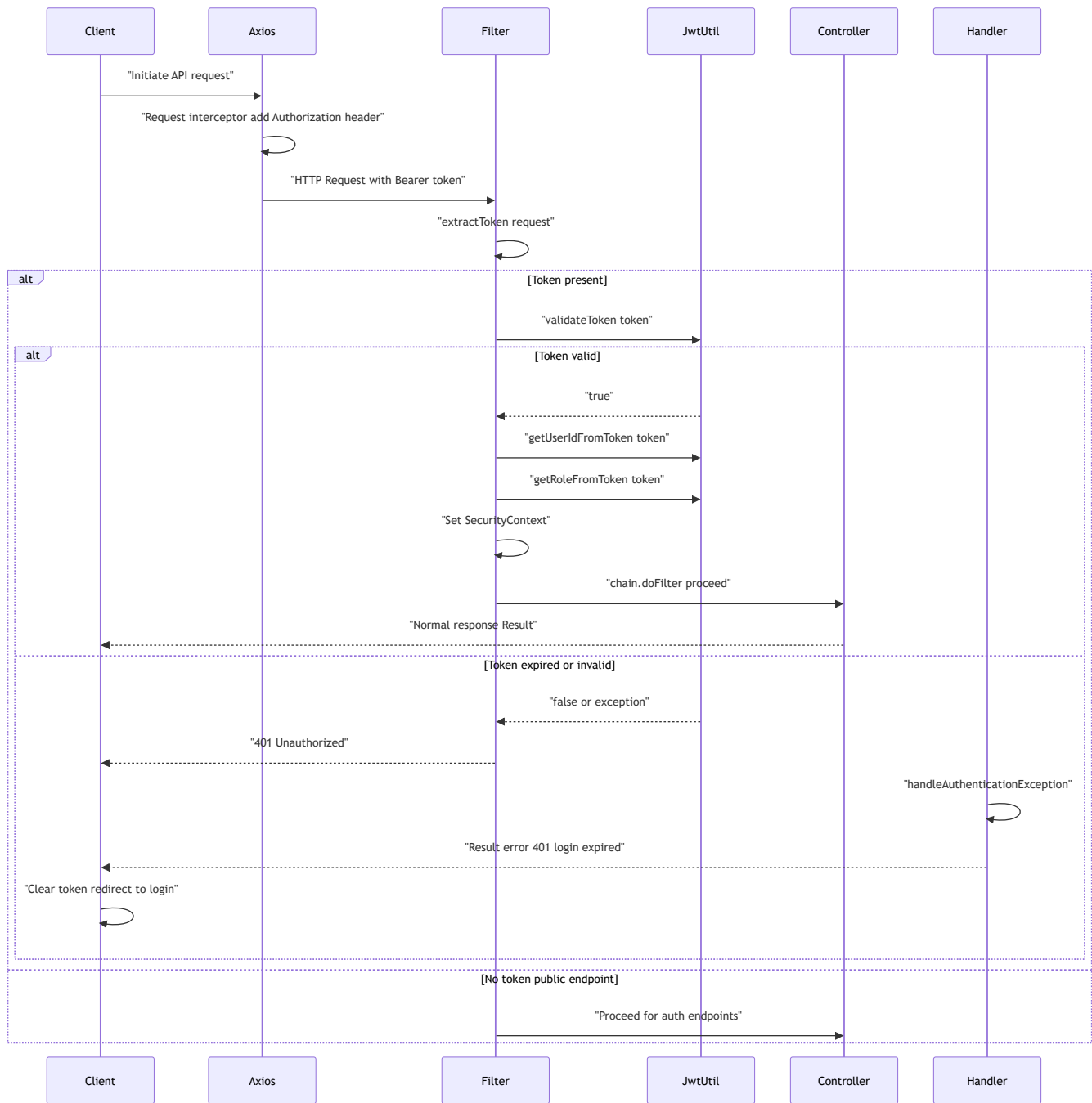


5.5.3 核心时序图

流程 A：患者登录 + JWT 认证



流程 B：JWT 鉴权拦截流程



流程 C：全局异常处理

```

Parse error on line 18:
... not logged in"      else Unknown error
^
Expecting 'SPACE', 'NL', 'participant', 'activate', 'deactivate', 'title', 'loop',
'end', 'opt', 'alt', 'par', 'note', 'ACTOR', got 'else'
  
```

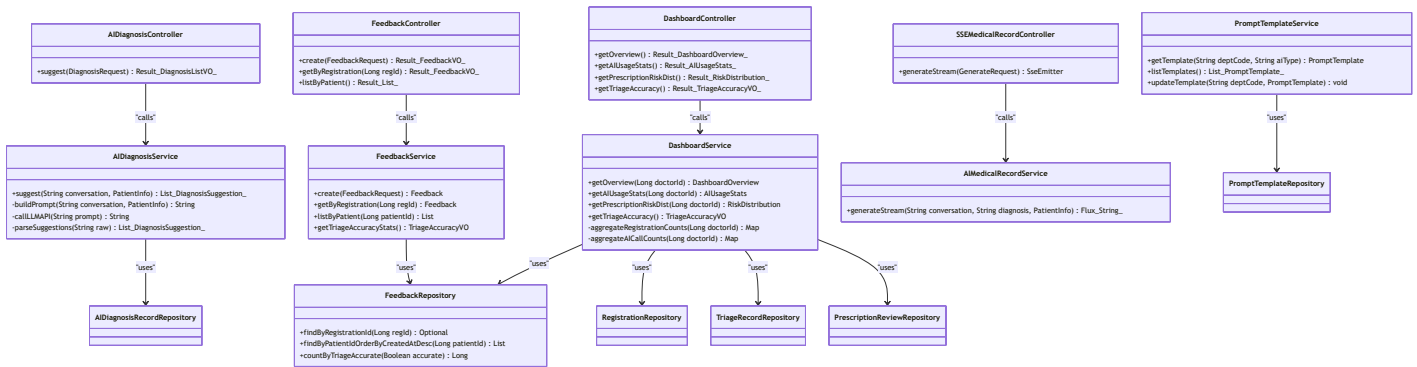
5.6 模块六：P1 进阶能力模块

5.6.1 模块概述

负责人：由 P0 模块先完成的成员认领

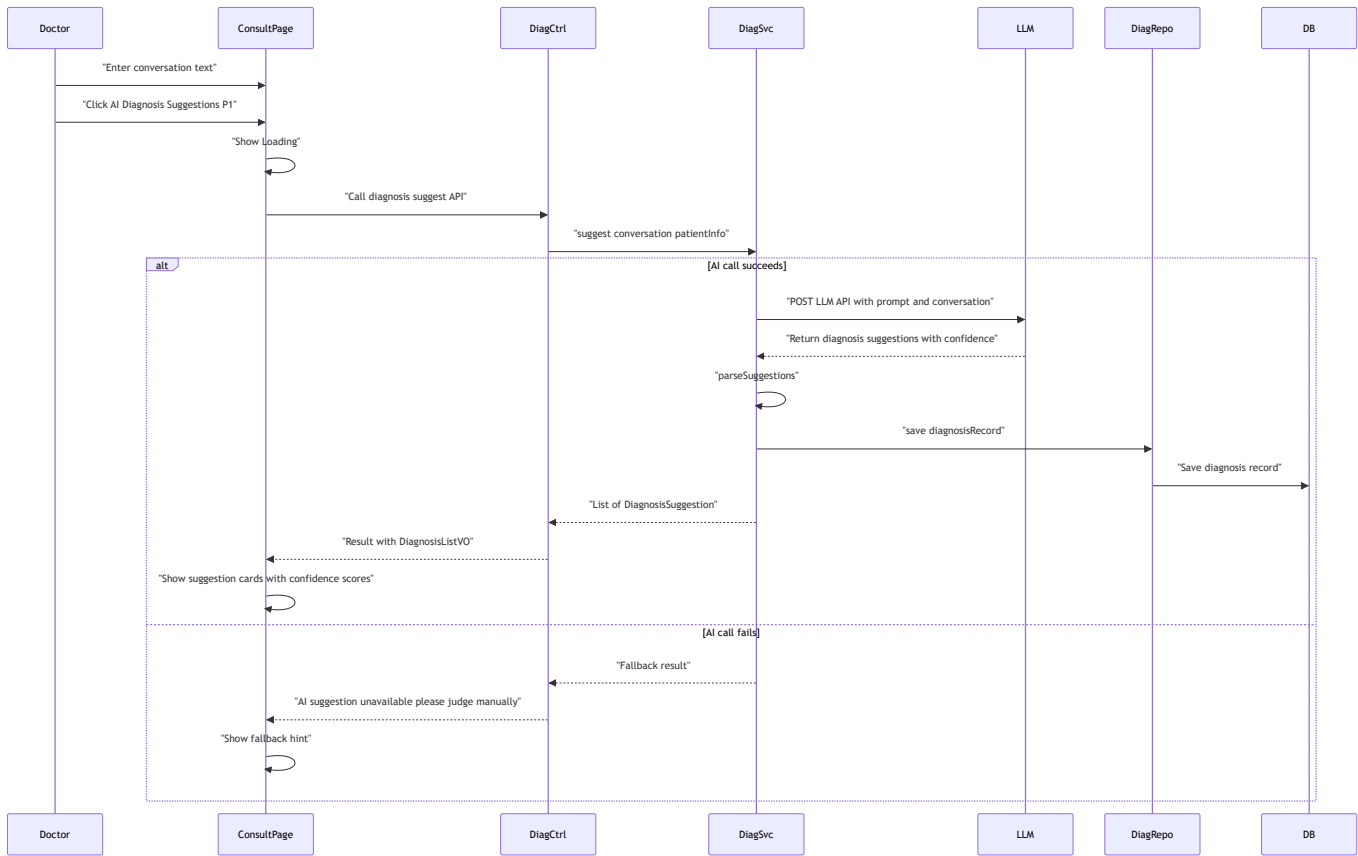
职责：覆盖 PRD 中全部 P1 能力——AI 诊疗建议、就诊评价与反馈、数据可视化看板、SSE 流式病历生成、Pinia 状态机模块化、可配置 Prompt 模板。WebSocket 实时通知和双端分离部署、微服务拆分属于 E04-E09 拓展，在本模块中做概要设计。

5.6.2 类图

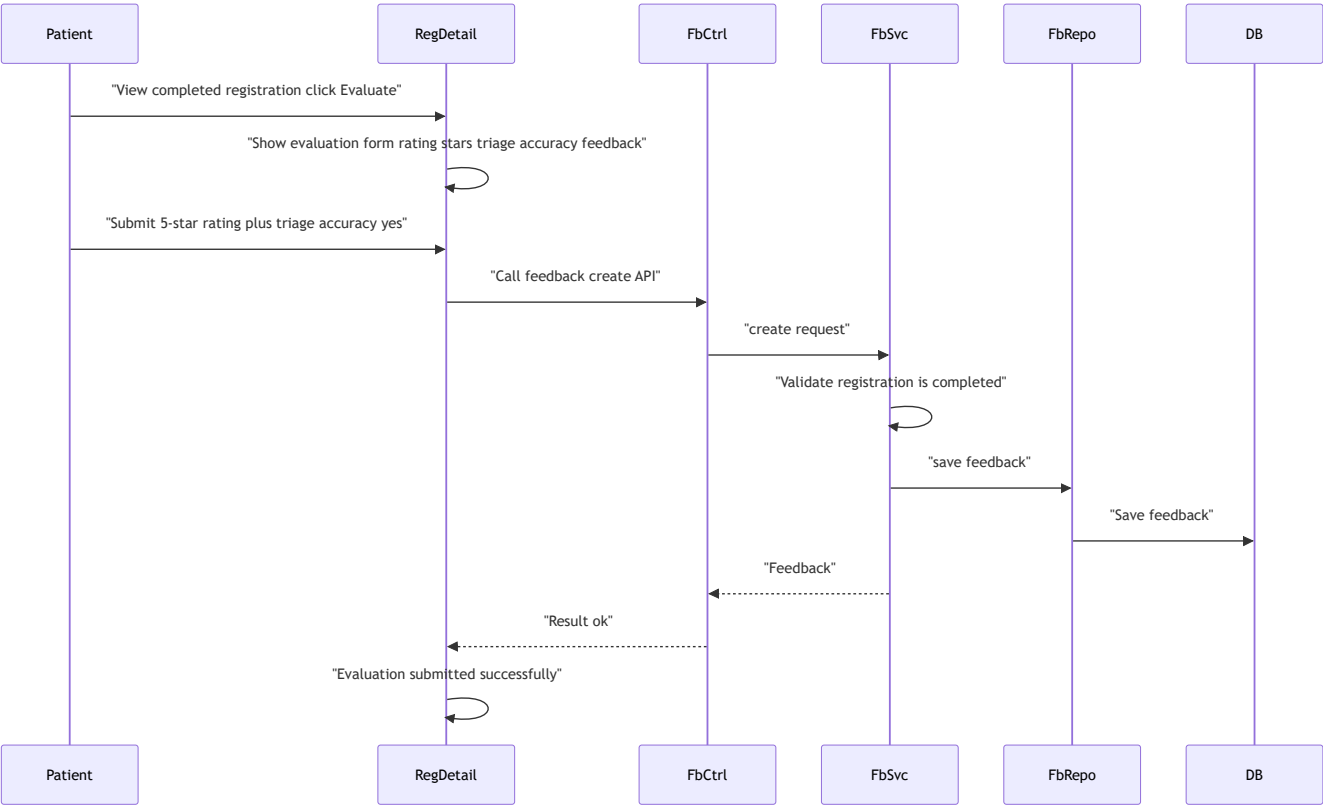


5.6.3 核心时序图

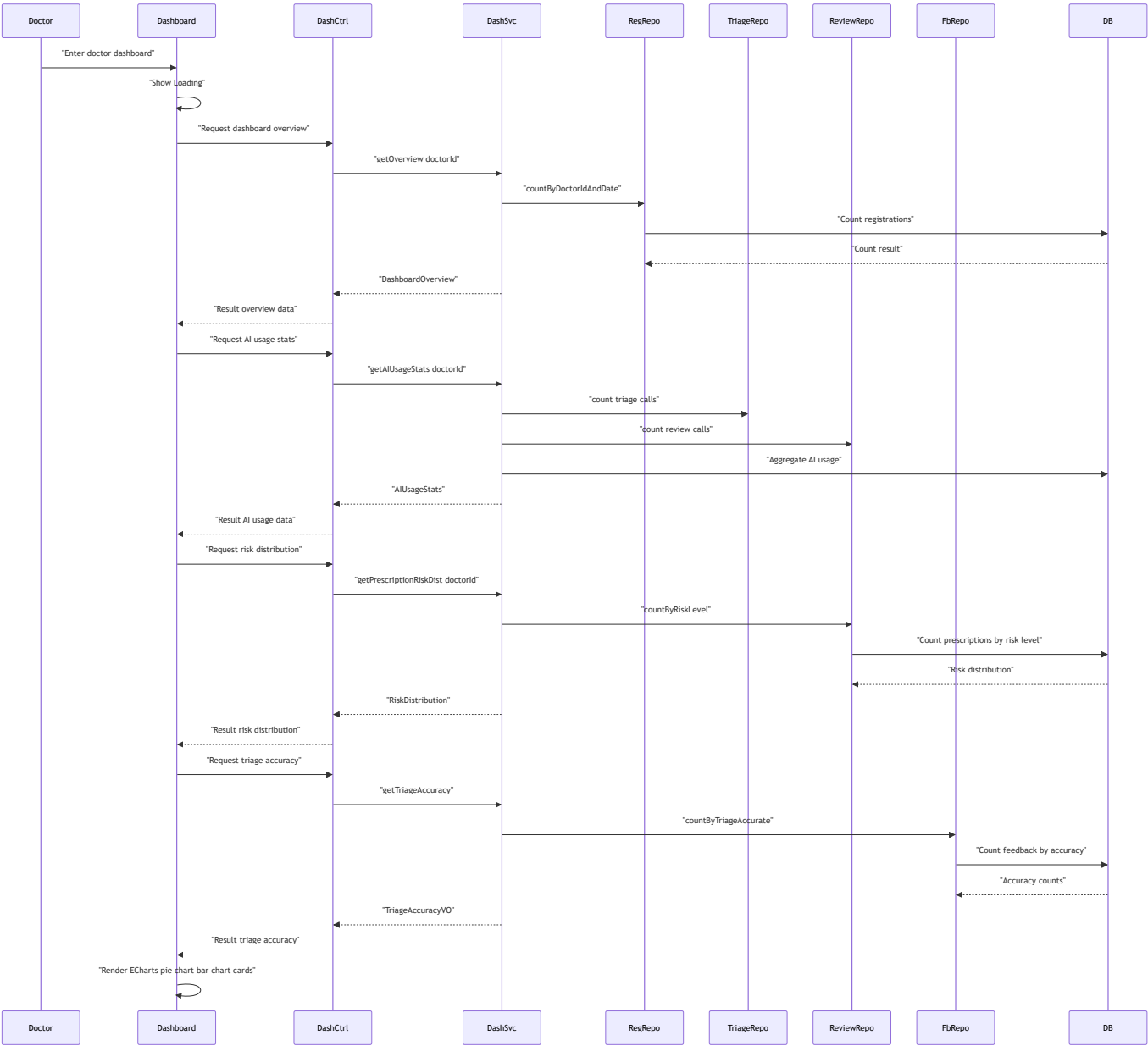
流程 A：AI 诊疗建议（P1）



流程 B：就诊评价（P1）



流程 C：ECharts 数据看板（P1）



5.6.4 其他 P1 能力概要设计

P1 能力	PRD 编号	设计要点
SSE 流式响应	E05	<code>AIMedicalRecordService.generateStream()</code> 返回 <code>Flux<String></code> （对应前端 <code>EventSource</code> ）；降级为普通 POST
Pinia 状态机	E06	新增 <code>useScheduleStore</code> 、 <code>useDrugStore</code> 、 <code>useDashboardStore</code> ；AI 请求状态统一由 <code>useAIStore</code> 管理（idle/loading/success/error）
Prompt 模板配置	E07	<code>PromptTemplateService</code> 按科室+AI 类型读取 YAML/DB 模板，支持变量 <code>{{dept}}</code> 、 <code>{{age}}</code> 替换
WebSocket 通知	E04	<code>WebSocketConfig + RiskAlertHandler</code> ，高风险时 <code>SimpMessagingTemplate.convertAndSend("/topic/risk-alert")</code>
双端分离部署	E08	<code>npm run build</code> → dist/ → Nginx； <code>mvn package</code> → .jar → <code>java -jar</code> ；Nginx <code>proxy_pass</code> 转发 <code>/api/</code>
微服务拆分	E09	AI Service 独立为 Spring Boot 应用；Nacos 注册中心 + Spring Cloud Feign 远程调用

6. API 接口总览

模块五——认证与患者管理

方法	路径	说明	鉴权
POST	<code>/api/auth/patient/register</code>	患者注册	否
POST	<code>/api/auth/patient/login</code>	患者登录	否
POST	<code>/api/auth/doctor/login</code>	医生登录	否
POST	<code>/api/auth/admin/login</code>	管理员登录	否
GET	<code>/api/patient/info</code>	获取患者信息	PATIENT
PUT	<code>/api/patient/info</code>	更新患者信息	PATIENT

模块一——智能分诊与挂号

方法	路径	说明	鉴权
POST	<code>/api/triage/consult</code>	AI 智能分诊	PATIENT
GET	<code>/api/triage/history</code>	分诊历史	PATIENT

方法	路径	说明	鉴权
GET	/api/departments	科室列表（公开）	否
GET	/api/departments/{id}	科室详情（公开）	否
GET	/api/doctors	医生列表按科室筛选（公开）	否
GET	/api/doctors/{id}	医生详情（公开）	否
GET	/api/schedules/available	可用号源	PATIENT
POST	/api/registration/create	创建挂号（扣减号源）	PATIENT
GET	/api/registration/list	我的挂号列表	PATIENT
PUT	/api/registration/cancel/{id}	取消挂号（释放号源）	PATIENT

模块二——AI 病历生成

方法	路径	说明	鉴权
POST	/api/medical-record/generate	AI 生成病历	DOCTOR
POST	/api/medical-record/save	保存病历并标记挂号完成	DOCTOR
GET	/api/medical-record/{id}	病历详情	DOCTOR/PATIENT
GET	/api/medical-record/list/patient/{id}	患者病历列表	DOCTOR/PATIENT
GET	/api/medical-record/list/doctor	医生病历列表	DOCTOR
GET	/api/medical-record/search	搜索病历	DOCTOR

模块三——处方审核与开具

方法	路径	说明	鉴权
POST	/api/prescription/create	创建处方	DOCTOR
POST	/api/prescription/check	AI 处方审核（本地规则+LLM）	DOCTOR
GET	/api/prescription/list/patient/{id}	患者处方列表	DOCTOR/PATIENT
GET	/api/prescription/list/doctor	医生处方列表	DOCTOR
GET	/api/prescription/{id}	处方详情	DOCTOR/PATIENT
GET	/api/prescription/{id}/review	审核详情	DOCTOR/PATIENT

方法	路径	说明	鉴权
GET	/api/drugs/search	医生端搜索药品（仅上架）	DOCTOR

模块四——管理端

方法	路径	说明	鉴权
GET/POST/PUT/PATCH	/api/admin/departments/**	科室 CRUD	ADMIN
GET/POST/PUT/PATCH	/api/admin/doctors/**	医生 CRUD	ADMIN
GET/POST/PUT/PATCH	/api/admin/schedules/**	排班号源管理 含批量	ADMIN
GET/POST/PUT/PATCH	/api/admin/drugs/**	药品库管理	ADMIN
GET/POST/PUT/PATCH	/api/admin/prescription-rules/**	处方规则管理	ADMIN
GET/PUT	/api/admin/ai-config/**	AI 配置管理	ADMIN
GET	/api/admin/ai-records/triage	AI 分诊记录	ADMIN
GET	/api/admin/ai-records/medical-record	病历生成记录	ADMIN
GET	/api/admin/ai-records/prescription-review	处方审核记录	ADMIN

模块六——P1 进阶能力

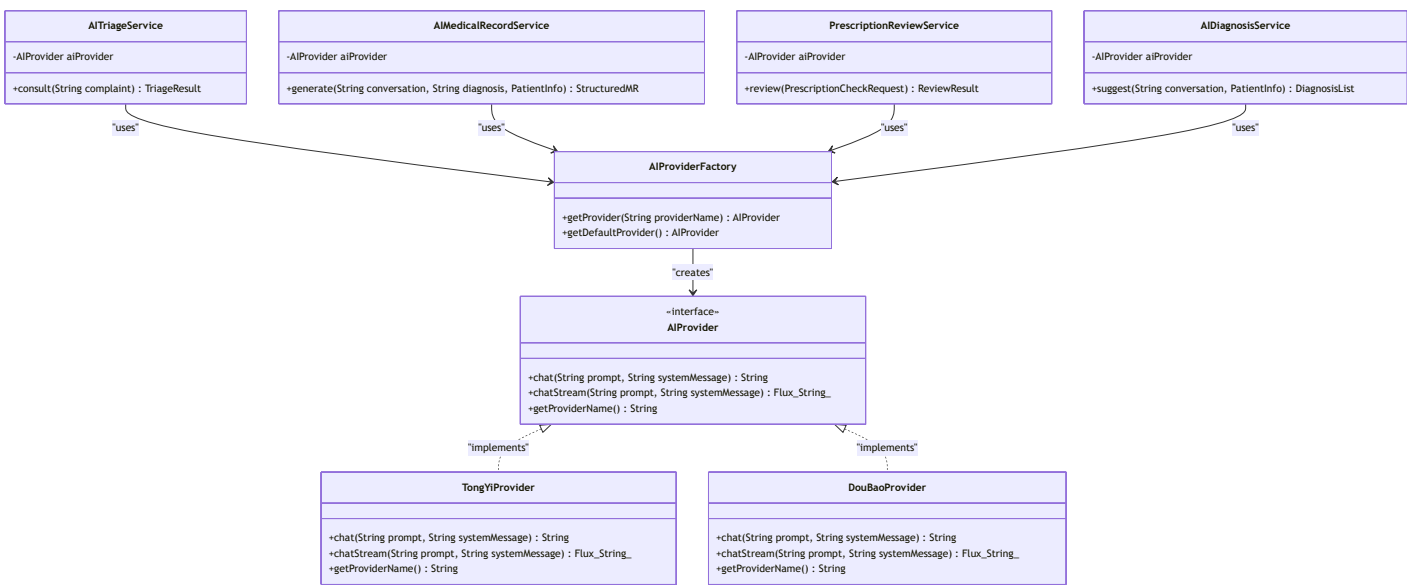
方法	路径	说明	鉴权
POST	/api/diagnosis/suggest	AI 诊疗建议 P1	DOCTOR
POST	/api/feedback/create	提交就诊评价 P1	PATIENT
GET	/api/feedback/registration/{id}	查询评价 P1	PATIENT
GET	/api/dashboard/overview	看板概览 P1	DOCTOR
GET	/api/dashboard/ai-usage	AI 使用率统计 P1	DOCTOR
GET	/api/dashboard/risk-distribution	处方风险分布 P1	DOCTOR
GET	/api/dashboard/triage-accuracy	分诊准确率 P1	DOCTOR
GET	/api/medical-record/generate-stream	SSE 流式病历生成 P1	DOCTOR

统一响应格式

```
{
  "code": 200,
  "message": "success",
  "data": {...},
  "timestamp": 1718323200000
}
{"code": 400, "message": "validation error", "data": null, "timestamp": 1718323200000}
{"code": 401, "message": "login expired please re-login", "data": null, "timestamp": 1718323200000}
{"code": 500, "message": "internal server error", "data": null, "timestamp": 1718323200000}
```

7. AI 服务集成设计

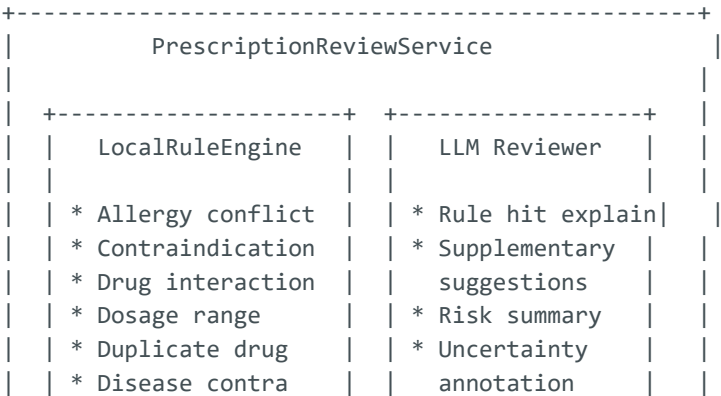
7.1 AI Provider 抽象层

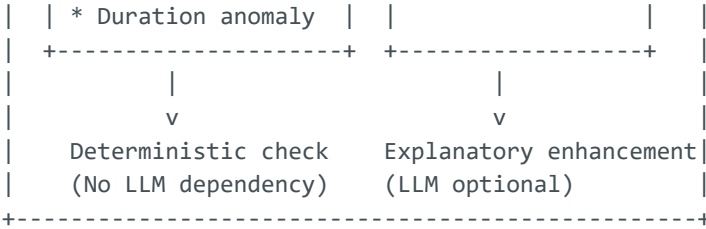


关键设计：

- **AIProvider** 接口定义统一 **chat / chatStream** 方法，所有供应商对业务层透明
- **AIProviderFactory** 按 **AIConfig** 表配置动态创建实例
- 切换供应商：修改 **AIConfig.provider** 字段值 + 重启即可
- 前端永不接触 API Key

7.2 处方审核双引擎架构





8. 安全设计

层次	措施	说明
传输层	HTTPS 生产 / HTTP 开发	Nginx 反向代理 + SSL
认证层	JWT + BCrypt	Token 24h 有效期，密码 BCrypt
鉴权层	基于角色的访问控制 RBAC	PATIENT / DOCTOR / ADMIN
数据层	API Key AES 加密存储	前端不接触 Key
应用层	@Valid + JPA 参数绑定	防 SQL 注入
前端	localStorage + Axios 拦截器	401 自动跳登录

文档结束。下一步：编写各模块实现计划文档，分配任务到人。